

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Dossier de validation **WAC (Windows Artefacts Collector)**

Logiciel de prélèvement d'artefacts Windows
pour l'investigation numérique



ARTEFACTS
SYSTÈME



FICHIERS
UTILISATEUR



PREUVES
NUMÉRIQUES

UN OUTIL AU SERVICE
DE L'INVESTIGATION
NUMÉRIQUE

WAC — Windows Artefacts Collector

[Dossier de validation](#)

Version du logiciel	1.3.1
Édition du document	septembre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC) École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document recense les contrôles qui établissent la qualité de WAC : ceux que WAC exécute lui-même à chaque collecte, les programmes de test qui confrontent ses décodeurs à une référence, et les contrôles automatiques de ses sorties. Pour chacun, il dit ce qui est comparé, à quoi, le résultat de référence et les défauts qu'il a déjà pris. Il dit aussi ce que ces contrôles ne garantissent pas. La façon de lancer chaque test est dans le *Guide des tests*.

Table des matières

1	La démarche	2
1.1	Pourquoi un JSON valide ne prouve rien	2
1.2	Quatre niveaux de contrôle	2
1.3	Les règles de développement qui s’y rattachent	2
2	Les contrôles intégrés à WAC	4
2.1	Avant toute extraction	4
2.2	Lecture brute du volume NTFS	4
2.3	Ruches du registre	4
2.4	Intégrité des pièces : la consigne	5
2.5	Authenticité des binaires	5
2.6	Journaux d’événements	6
2.7	Décodage des artefacts	6
3	Les programmes de test	7
3.1	Harnais de format	7
3.2	Conversions : le détail de system_conversions_test	8
3.3	Harnais de robustesse	9
4	Le cycle automatique et ses contrôles	10
4.1	Le cycle	10
4.2	Les contrôles de check-json.py	10
4.3	Résultat de référence	12
5	Les recoupements sur une collecte réelle	13
6	Ce que ces contrôles garantissent	14
6.1	Garanties	14
6.2	Limites	14

La démarche

1.1 Pourquoi un JSON valide ne prouve rien

WAC produit des fichiers JSON destinés à une expertise. Qu'ils soient valides ne dit presque rien de leur justesse : un champ lu au mauvais décalage, une date interprétée dans le mauvais fuseau, un nom décodé dans la mauvaise page de code, une décompression fausse donnent tous un JSON parfaitement valide, et faux. Les défauts les plus graves relevés dans WAC étaient tous de cette sorte. Aucun ne se voyait à l'exécution; chacun n'est apparu qu'en confrontant une valeur à une **source indépendante**.

La validation de WAC repose donc sur ce principe : toute donnée qui peut être recoupée l'est, par un moyen qui ne partage rien avec le code vérifié.

1.2 Quatre niveaux de contrôle

Les contrôles intégrés à WAC (chapitre 2) : exécutés à chaque collecte, sur la machine examinée. Ils refusent une donnée incohérente plutôt que de la publier, et attestent l'intégrité des pièces prélevées.

Les programmes de test (chapitre 3) : quatorze exécutables qui confrontent chaque décodeur à une référence (la fonction de Microsoft qu'il remplace, une norme publiée, une autre implémentation), ou qui prouvent qu'un analyseur ne lit jamais hors de son tampon.

Le cycle automatique et ses contrôles (chapitre 4) : une collecte complète dans une machine virtuelle Windows 11, puis `check-json.py`, qui vérifie la validité des sorties et les confronte entre elles et à des sources externes.

Les recoupements sur une collecte réelle (chapitre 5) : ce qu'une machine virtuelle neuve ne contient pas (journaux anciens, fichiers fragmentés, noms exotiques) est vérifié sur les données d'une vraie machine, traitées en local.

1.3 Les règles de développement qui s'y rattachent

1. **Un correctif s'accompagne du test qui aurait pris le défaut**, et ce test est vérifié dans les deux sens : il passe sur le code corrigé, et il échoue quand on réintroduit le défaut. Un test qui n'a jamais été vu échouer ne prouve rien.
2. **La compilation se fait sans avertissement**, options `-Wall -Wextra` activées. Les avertissements ont d'ailleurs servi de liste de travaux : chacun signalait un champ

décodé puis perdu.

3. **Après toute modification, la nouvelle collecte est comparée à la précédente,** artefact par artefact : un artefact passé de 1 522 entrées à 573 n'allume aucun voyant, seule la comparaison le montre.
4. **Toute entrée est hostile,** y compris un fichier du disque examiné : chaque lecture est bornée par la taille réelle du tampon, jamais par une longueur annoncée dans la donnée.
5. **Un champ sans valeur n'est pas émis** : une clé vide ou nulle se lirait comme une donnée relevée, ou comme une lecture ratée.

Les contrôles intégrés à WAC

Ces contrôles font partie du programme de collecte. Ils s'exécutent à chaque collecte et leur résultat est consigné dans `investigation.json`, opération par opération, avec son code de retour et la trace qu'elle laisse sur le système examiné.

2.1 Avant toute extraction

- version de Windows (10 ou ultérieure) et droits d'administrateur effectifs (jeton élevé);
- emplacement de collecte utilisable et place disponible, vérifiés *avant* la première extraction : une collecte interrompue faute de place laisserait une consigne incomplète.

2.2 Lecture brute du volume NTFS

Les fichiers verrouillés (ruches, journaux) sont lus directement sur le volume, sans passer par le système de fichiers de Windows. Chaque structure lue est vérifiée avant d'être utilisée :

- signature FILE de chaque enregistrement de la \$MFT et INDX de chaque bloc d'index, puis vérification des *fixups* (tableau de séquence de mise à jour) : chaque tranche de 512 octets doit se terminer par le numéro de séquence de l'enregistrement. Un enregistrement *déchiré* par une écriture interrompue, mélange de deux versions, est refusé au lieu d'être lu comme une donnée plausible;
- décompression LZNT1 et WOF bornée par la taille attendue de chaque unité : une unité qui ne produit pas le nombre d'octets prévu est une erreur, pas une donnée tronquée en silence;
- taille extraite comparée à la taille déclarée par l'attribut \$DATA : leur écart, publié dans le manifeste, révèle une extraction tronquée qu'une empreinte seule ne montrerait pas.

2.3 Ruches du registre

WAC lit les ruches avec son propre lecteur, lié dans l'exécutable, et non avec une bibliothèque de la machine examinée.

- bloc de base : signature regf, somme de contrôle (OU exclusif des 127 premiers mots), numéros de séquence primaire et secondaire. Une ruche dont les journaux n'ont pas

été rejoués est **refusée** : lue telle quelle, elle donnerait des clés périmées sans aucune erreur ;

- rejeu des journaux de transactions (.LOG1/.LOG2) : signature et chaînage des séquences de chaque entrée ; une entrée hors chaîne est écartée et comptée ; un journal d'annulation est écrit avant toute modification, pour pouvoir reconstituer la copie brute ;
- chaque décalage lu dans la ruche est vérifié contre la taille réelle du fichier, et une sous-clé qui pointerait vers l'une de ses ancêtres est refusée : un parcours ne peut pas boucler sur une ruche forgée.

2.4 Intégrité des pièces : la consigne

Toute pièce extraite est déposée dans une *consigne* qui n'est plus jamais rouverte en écriture ; tout le travail se fait sur une seconde copie.

- trois empreintes par pièce (MD5, SHA-1, SHA-256), calculées pendant la lecture du volume ;
- la copie de travail est vérifiée par son SHA-256 contre ce qui a été lu sur le volume, et non contre une relecture de la consigne ;
- le manifeste qui identifie les pièces est scellé par un second fichier portant son empreinte : un manifeste retouché se détecte ;
- un échec d'extraction est consigné lui aussi : une pièce absente du manifeste se lirait comme jamais cherchée.

Un tiers peut vérifier la consigne sans WAC, avec les scripts `verify-exhibits.ps1` (messages en anglais) et `verifier-consigne.ps1` (messages en français), livrés avec la documentation utilisateur : ils contrôlent le sceau puis l'empreinte de chaque pièce.

2.5 Authenticité des binaires

Avec l'option `--binary`, WAC prélève les binaires cités par les artefacts, sauf ceux dont il établit qu'ils sont signés par Microsoft. Cette décision repose sur une vérification complète, faite par WAC lui-même : signature RSA de chaque catalogue de sécurité, chaîne de certificats jusqu'à une racine Microsoft, empreinte Authenticode du fichier retrouvée dans un catalogue vérifié, ou signature intégrée. Un fichier dont l'authenticité n'est pas établie est prélevé : le doute conduit à collecter, jamais à écarter.

2.6 Journaux d'événements

Signature du fichier, de chaque bloc (*chunk*) et de chaque enregistrement. Tous les blocs physiques du fichier sont parcourus, et pas seulement ceux que l'en-tête déclare : un journal mal fermé livre ainsi les enregistrements qu'il ne compte pas. Un bloc illisible est signalé et sauté ; il n'interrompt plus la lecture du fichier.

2.7 Décodage des artefacts

- toute lecture est bornée par la taille du tampon ; un nombre d'éléments déraisonnable dans une liste de valeurs (au-delà de 65 536) est refusé plutôt que parcouru ;
- une date impossible (mois 13, heure 25) ou hors de la plage d'un FILETIME donne une date absente, donc non émise, et jamais une valeur quelconque ;
- chaque date locale porte le décalage en vigueur à sa date, selon les règles d'heure d'été de la machine examinée, année par année.

Les programmes de test

Quatorze exécutables, exclus de la compilation de WAC par leur suffixe `_test.cpp` et construits par `build-windows.sh --test`. Chacun vise un composant et le confronte à un juge qui ne partage rien avec lui.

3.1 Harnais de format

Programme	Juge	Résultat de référence
sha_test	vecteurs de FIPS 180-4, implémentation indépendante	SHA-1 et SHA-256 conformes, y compris sur les longueurs 55 à 128 octets qui exercent le remplissage
lznt1_test	compresseur de Microsoft (RtlCompressBuffer)	16 unités compressées sur 16 conformes, 1 048 576 octets identiques
xpress_test	compresseur de Microsoft	137 blocs compressés sur 137 conformes, octet pour octet
wevt_test	fichiers d'un vrai fournisseur d'événements	chaîne complète de production des messages vérifiée, marques de substitution comprises
event_messages_test	ruche SOFTWARE et binaires de la machine	même chaîne, étape par étape, à partir de la ruche
evtx_test	python-evtx, implémentation indépendante	enregistrement par enregistrement; chaîne complète identique en flux et en mémoire (pic de 26 Mo contre 1 281 Mo)
offline_registry_test	offreg.dll de Microsoft	18 ruches, dont dix d'une machine réelle : 1 273 652 clés et 2 456 139 valeurs identiques
system_conversions_test	fonctions Windows remplacées	9 661 284 comparaisons, aucun écart (détail ci-dessous)
consigne_test	la consigne elle-même, avant et après	consigne identique octet pour octet après modification de la copie de travail; sceau conforme

Programme	Juge	Résultat de référence
ntfs_fixup_test	enregistrements construits, vérification indépendante	enregistrements sains restitués à l'octet, enregistrements déchirés et tableaux malformés refusés, 20 000 enregistrements aléatoires contre une page interdite : 44 610 contrôles, aucun échec
raw_hive_test	reg load de Windows	la ruche extraite est refusée avant réparation et acceptée après : le test échoue dans les deux sens s'il le faut
authenticode_test	Get-Authenticode Signature	2 233 binaires : 2 126 authentifiés par les deux, 99 prélevés par les deux, aucun fichier accepté par WAC et refusé par Windows; 462 scripts PowerShell et 11 scripts WSH conformes

3.2 Conversions : le détail de system_conversions_test

WAC ne dépend que de cinq bibliothèques système (ADVAPI32, KERNEL32, msvcrt, Secur32, WTSAPI32), nécessaires pour interroger le système en marche. Les conversions qu'il demandait autrefois à d'autres bibliothèques de la machine examinée sont faites par son propre code, et ce programme les confronte aux fonctions de Windows qu'elles remplacent :

- GUID, SID dans les deux sens, dates OLE, noms de propriétés (table de 1 698 entrées générée depuis le schéma de Windows);
- dates FAT, sur toutes les dates possibles;
- heure d'été : décalage dans les deux sens sur les **141 fuseaux** de Windows, avec leurs règles année par année, à chaque transition de 1980 à 2035 (heure sautée, heure répétée, changement de règle au 1^{er} janvier);
- lecture de la clé TimeZoneInformation, comparée à GetTimeZoneInformation;
- règles de fuseau forgées au hasard : aucun débordement.

Écarts voulus, et documentés : WAC refuse un SID écrit que Windows tronquerait, et une date OLE non numérique pour laquelle Windows rend une heure dénuée de sens.

3.3 Harnais de robustesse

Un analyseur lit à des positions que le fichier déclare lui-même. Sur un fichier tronqué ou forgé, une lecture hors tampon ne plante pas d'elle-même : elle lit la mémoire voisine et la publie, dans un JSON valide. Les harnais de robustesse la font planter : chaque entrée est placée contre une page mémoire interdite, une fois à sa fin, une fois à son début. Un seul octet lu hors de l'entrée arrête le test. Un chien de garde arrête aussi un analyseur qui ne termine pas.

Chaque fichier est analysé entier, dans ses troncatures, et en copies corrompues au hasard (graine fixe : un échec se reproduit à l'identique).

Programme	Fichiers d'une machine réelle	Entrées analysées
lnk_test	220 raccourcis	546 214
parsers_test	48 listes de sauts automatiques	175 585
	23 listes de sauts personnalisées	79 036
	307 Prefetch	1 246 491
	4 ruches du registre	16 207

Résultat : aucune lecture hors tampon, aucune boucle sans fin.

Le cycle automatique et ses contrôles

4.1 Le cycle

`vmtest/run-wac-test.sh` exécute, sans aucune intervention, une collecte complète dans une machine virtuelle Windows 11 (vrai NTFS, vrai registre) :

1. avec `--build`, compilation sans avertissement de WAC et des programmes de test;
2. validation de l'extraction brute d'une ruche : `reg load` doit la refuser avant réparation et l'accepter après;
3. exécution de WAC avec les droits SYSTEM; le code de retour est vérifié, et la trace est cherchée pour toute exception non traitée;
4. rapatriement des sorties, du journal de collecte et de la liste des fichiers réellement présents dans la consigne;
5. exécution de `check-json.py`.

4.2 Les contrôles de `check-json.py`

Le script vérifie d'abord la validité JSON de chaque fichier et la forme des chemins Windows (un double échappement se voit après analyse). Il confronte ensuite des valeurs calculées par WAC à des valeurs indépendantes : une autre donnée de la même collecte, ou une source externe. Chaque contrôle ci-dessous a été écrit après un défaut réel, qu'il a pris.

Contrôle	Défaut qu'il a pris
<i>Dates</i>	
couples <code>X / XUtc</code> , listes comprises, désignant le même instant	double décalage horaire (sessions, raccourcis, date d'installation); heures d'exécution des Prefetch étiquetées <code>+02:00</code> au lieu de UTC (1 273 valeurs fausses de 2 h)
décalage de chaque date locale / base de fuseaux IANA	toutes les dates étiquetées avec le décalage du jour de collecte (106 dates d'hiver à <code>+02:00</code>); dates de changement d'heure de la ruche SYSTEM lues dans le mauvais format

Contrôle	Défaut qu'il a pris
aucune session antérieure au démarrage du système	un décalage de deux heures sur les sessions ; plus tard, une heure de démarrage en retard de 3,5 s
heure de démarrage / événement Kernel-General 12	deux sources indépendantes du même instant : 0,00 s d'écart
date ShimCache / date NTFS du fichier dans le manifeste	une date UTC traitée comme locale
aucun événement postérieur à la collecte	décalage ou mauvaise lecture d'une date d'événement
<i>Décodage</i>	
empreinte d'un Prefetch / suffixe de son nom de fichier	65 empreintes fausses sur 270 (hexadécimal sans remplissage)
chemins \VOLUME{guid} des Prefetch résolus	20 066 chemins sur 20 066 non résolus
entrées DestList / raccourcis des listes de sauts	les 15 premières entrées de chaque liste perdues (573 raccourcis publiés sur 1 522)
nom de machine des événements / OperatingSystem.json	contrôle du décodage BinXML, sur deux sources sans lien
identifiants d'enregistrement uniques par journal	relecture d'un bloc périmé
références %%nnnn non résolues dans les messages	fichier de paramètres d'un fournisseur non chargé (3 780 messages)
RID retrouvé à la fin du SID reconstruit	lecture de la base SAM
états et types des services, présence des pilotes	filtres d'énumération confondus avec des états
aucun processus ne porte WAC.exe parmi ses modules	modules de l'outil attribués au processus Idle
références \$MFT plausibles	décodage des blocs d'extension des raccourcis
<i>Consigne et rejeu</i>	
sceau / empreinte réelle du manifeste	retouche du manifeste
trois empreintes par pièce, comptes cohérents	pièce non identifiée
fichiers présents dans la consigne / manifeste	209 pièces ajoutées après le scellement, que tous les autres contrôles laissaient passer
aucune ruche à la fois rejouée et réparée	rejeu qui n'aligne pas les séquences

Contrôle	Défaut qu'il a pris
un journal d'annulation pour chaque jeu	copie brute impossible à reconstituer

4.3 Résultat de référence

Dernier cycle, sur la version 1.3.1 (Windows 11, fuseau de Paris) :

- 23 fichiers JSON valides sur 23 produits (le 24^e, USBSTOR, est absent : la machine virtuelle n'a jamais vu de clé USB);
- 26 contrôles croisés passés, aucun échec, dont 4 045 dates locales confrontées à la base IANA et 145 645 événements décodés;
- 1 336 pièces dans la consigne, exactement celles du manifeste, sceau conforme.

Les recoupements sur une collecte réelle

Une machine virtuelle neuve ne produit ni fragmentation, ni listes de sauts anciennes, ni noms de fichiers exotiques, ni journaux abîmés. Plusieurs défauts ne se sont révélés que sur une vraie machine, confrontée à une source indépendante :

Recoupement	Ce qu'il a montré
listes de sauts / olefile	les flux s'appellent « 1 » à « f », et non « 01 » à « 0f »
cible d'un raccourci / entrée DestList	une page de code fausse et un chemin Unicode ignoré
journaux / python-evtx	un bloc de signature fausse arrêta la lecture de tout le fichier (14 enregistrements lus sur 270)
blocs EVTX / leur CRC32	contrôle ponctuel : les 1 505 blocs des 404 journaux de la machine de test sont intègres
ruches / offreg.dll	1 273 652 clés identiques
binaires / Get-AuthenticodeSignature	aucune authentification abusive
ancienne sortie / nouvelle	décodage des mêmes fichiers par le code courant, comparé après normalisation

À retenir

Une collecte réelle contient des données personnelles. Elle se traite en local ; une copie temporaire dans la machine virtuelle ou un dossier de travail est effacée après usage, et rien n'en entre dans le dépôt des sources.

Ce que ces contrôles garantissent

6.1 Garanties

- **Intégrité des pièces** : chaque pièce est identifiée par trois empreintes, la copie de travail est vérifiée contre la lecture du volume, le manifeste est scellé, et la consigne n'est jamais modifiée (`consigne_test`, contrôles du cycle).
- **Aucune écriture sur le système examiné**, hors les traces inévitables d'une collecte sur un système en marche, toutes consignées dans `investigation.json` avec la trace attendue.
- **Exactitude des décodeurs** : pour ceux qui ont une référence (lecteur de ruches, décompressions, empreintes, conversions, fuseaux horaires, authenticité des binaires), résultats identiques à la référence sur les volumes indiqués.
- **Robustesse** : aucun analyseur de raccourcis, listes de sauts, Prefetch ou ruches ne lit hors de son tampon ni ne boucle, sur plus de deux millions d'entrées tronquées ou corrompues.
- **Cohérence des sorties** : chaque recoupement possible entre deux artefacts, ou entre un artefact et une source externe, est vérifié à chaque cycle.
- **Indépendance vis-à-vis de la machine examinée** : WAC n'emprunte à celle-ci aucune bibliothèque pour interpréter les preuves.

6.2 Limites

Limite

Ces contrôles ne prouvent pas l'absence de défaut. Ils prouvent que les défauts qu'ils visent sont absents, sur les données qu'ils ont vues.

- **Complétude** : un artefact vide ne se distingue pas d'une machine sans trace. Seule la comparaison entre deux collectes, ou avec une machine dont on connaît le contenu, la mesure.
- **Données que la machine de test ne contient pas** : clés USB, appareils MTP, fichiers éclatés sur plusieurs enregistrements de la `$MFT ($ATTRIBUTE_LIST)`, très gros volumes. Ils sont à valider sur des systèmes réels.
- **Artefacts sans référence** : quand aucun outil indépendant ne lit un format, sa justesse

repose sur les recoupements internes et sur la documentation publique du format.

- **Fuseaux horaires** : la confrontation à la base IANA couvre les fuseaux dont les règles concordent avec celles de Windows sur la période vérifiée (Europe de l'Ouest, États-Unis, UTC); les autres sont vérifiés contre Windows seulement.
- **Compilation MSVC** : le projet Visual Studio est maintenu, mais les contrôles sont faits sur la version compilée par MinGW, celle qui est livrée.